arXiv:1607.06450v1 [stat.ML] 21 Jul 2016

Layer Normalization

Jimmy Lei Ba
University of Toronto
jimmy@psi.toronto.edu

Jamie Ryan Kiros
University of Toronto
rkiros@cs.toronto.edu

Geoffrey E. Hinton
University of Toronto
and Google Inc.
hinton@cs.toronto.edu

Abstract

Training state-of-the-art, deep neural networks is computationally expensive. One way to reduce the training time is to normalize the activities of the neurons. A recently introduced technique called batch normalization uses the distribution of the summed input to a neuron over a mini-batch of training cases to compute a mean and variance which are then used to normalize the summed input to that neuron on each training case. This significantly reduces the training time in feed-forward neural networks. However, the effect of batch normalization is dependent on the mini-batch size and it is not obvious how to apply it to recurrent neural networks. In this paper, we transpose batch normalization into layer normalization by computing the mean and variance used for normalization from all of the summed inputs to the neurons in a layer on a *single* training case. Like batch normalization, we also give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity. Unlike batch normalization, layer normalization performs exactly the same computation at training and test times. It is also straightforward to apply to recurrent neural networks by computing the normalization statistics separately at each time step. Layer normalization is very effective at stabilizing the hidden state dynamics in recurrent networks. Empirically, we show that layer normalization can substantially reduce the training time compared with previously published techniques.

1 Introduction

Deep neural networks trained with some version of Stochastic Gradient Descent have been shown to substantially outperform previous approaches on various supervised learning tasks in computer vision [Krizhevsky et al., 2012] and speech processing [Hinton et al., 2012]. But state-of-the-art deep neural networks often require many days of training. It is possible to speed-up the learning by computing gradients for different subsets of the training cases on different machines or splitting the neural network itself over many machines [Dean et al., 2012], but this can require a lot of communication and complex software. It also tends to lead to rapidly diminishing returns as the degree of parallelization increases. An orthogonal approach is to modify the computations performed in the forward pass of the neural net to make learning easier. Recently, batch normalization [Ioffe and Szegedy, 2015] has been proposed to reduce training time by including additional normalization stages in deep neural networks. The normalization standardizes each summed input using its mean and its standard deviation across the training data. Feedforward neural networks trained using batch normalization converge faster even with simple SGD. In addition to training time improvement, the stochasticity from the batch statistics serves as a regularizer during training.

Despite its simplicity, batch normalization requires running averages of the summed input statistics. In feed-forward networks with fixed depth, it is straightforward to store the statistics separately for each hidden layer. However, the summed inputs to the recurrent neurons in a recurrent neural network (RNN) often vary with the length of the sequence so applying batch normalization to RNNs appears to require different statistics for different time-steps. Furthermore, batch normaliza-

arXiv:1607.06450v1 [stat.ML] 21 Jul 2016

层归一化

吉米·雷·巴
University of
多伦多
jimmy@psi.toronto.edu

Jamie Ryan Kiros多伦多大学
rkiros@cs.toronto.edu

杰弗里·E·辛顿多伦多大学与谷歌公司
hinton@cs.toronto.edu

摘要

Training state-of-the-art, deep neural networks is computationally expensive. One way to reduce the training time is to normalize the activities of the neurons. A recently introduced technique called batch normalization uses the distribution of the summed input to a neuron over a mini-batch of training cases to compute a mean and variance which are then used to normalize the summed input to that neuron on each training case. This significantly reduces the training time in feed-forward neural networks. However, the effect of batch normalization is dependent on the mini-batch size and it is not obvious how to apply it to recurrent neural networks. In this paper, we transpose batch normalization into layer normalization by computing the mean and variance used for normalization from all of the summed inputs to the neurons in a layer on a *single* training case. Like batch normalization, we also give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity. Unlike batch normalization, layer normalization performs exactly the same computation at training and test times. It is also straightforward to apply to recurrent neural networks by computing the normalization statistics separately at each time step. Layer normalization is very effective at stabilizing the hidden state dynamics in recurrent networks. Empirically, we show that layer normalization can substantially reduce the training time compared with previously published techniques.

1 引言

Deep neural networks trained with some version of Stochastic Gradient Descent have been shown to substantially outperform previous approaches on various supervised learning tasks in computer vision [Krizhevsky et al., 2012] and speech processing [Hinton et al., 2012]. But state-of-the-art deep neural networks often require many days of training. It is possible to speed-up the learning by computing gradients for different subsets of the training cases on different machines or splitting the neural network itself over many machines [Dean et al., 2012], but this can require a lot of communication and complex software. It also tends to lead to rapidly diminishing returns as the degree of parallelization increases. An orthogonal approach is to modify the computations performed in the forward pass of the neural net to make learning easier. Recently, batch normalization [Ioffe and Szegedy, 2015] has been proposed to reduce training time by including additional normalization stages in deep neural networks. The normalization standardizes each summed input using its mean and its standard deviation across the training data. Feedforward neural networks trained using batch normalization converge faster even with simple SGD. In addition to training time improvement, the stochasticity from the batch statistics serves as a regularizer during training.

尽管实现方式较为简单，批量归一化仍需要计算输入数据的各项统计量之和后的平均值。在结构深度固定的前馈神经网络中，为每一层隐藏层分别存储这些统计量是相当容易的。然而，在循环神经网络（RNN）中，输入到循环神经元的输入数据之和往往会随着序列长度的不同而变化，因此似乎需要为不同的时间步使用不同的统计量才能对RNN应用批量归一化。此外，批量归一化还无法用于在线学习任务，或是那些必须使用极小批量数据的庞大分布式模型中。

tion cannot be applied to online learning tasks or to extremely large distributed models where the minibatches have to be small.

This paper introduces layer normalization, a simple normalization method to improve the training speed for various neural network models. Unlike batch normalization, the proposed method directly estimates the normalization statistics from the summed inputs to the neurons within a hidden layer so the normalization does not introduce any new dependencies between training cases. We show that layer normalization works well for RNNs and improves both the training time and the generalization performance of several existing RNN models.

2 Background

A feed-forward neural network is a non-linear mapping from a input pattern $\mathbf{x}$ to an output vector y. Consider the lth hidden layer in a deep feed-forward, neural network, and let a^l be the vector representation of the summed inputs to the neurons in that layer. The summed inputs are computed through a linear projection with the weight matrix W^l and the bottom-up inputs h^l given as follows:

$$a_i^l = {w_i^l}^\top h^l \qquad h_i^{l+1} = f(a_i^l + b_i^l) \tag{1}$$

where $f(\cdot)$ is an element-wise non-linear function and w_i^l is the incoming weights to the ith hidden units and b_i^l is the scalar bias parameter. The parameters in the neural network are learnt using gradient-based optimization algorithms with the gradients being computed by back-propagation.

One of the challenges of deep learning is that the gradients with respect to the weights in one layer are highly dependent on the outputs of the neurons in the previous layer especially if these outputs change in a highly correlated way. Batch normalization [Ioffe and Szegedy, 2015] was proposed to reduce such undesirable "covariate shift". The method normalizes the summed inputs to each hidden unit over the training cases. Specifically, for the ith summed input in the lth layer, the batch normalization method rescales the summed inputs according to their variances under the distribution of the data

$$\bar{a}_i^l = \frac{g_i^l}{\sigma_i^l}\left(a_i^l - \mu_i^l\right) \qquad \mu_i^l = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[a_i^l\right] \qquad \sigma_i^l = \sqrt{\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\left(a_i^l - \mu_i^l\right)^2\right]} \tag{2}$$

where $\bar{a}_i^l$ is normalized summed inputs to the ith hidden unit in the lth layer and g_i is a gain parameter scaling the normalized activation before the non-linear activation function. Note the expectation is under the whole training data distribution. It is typically impractical to compute the expectations in Eq. (2) exactly, since it would require forward passes through the whole training dataset with the current set of weights. Instead, μ and σ are estimated using the empirical samples from the current mini-batch. This puts constraints on the size of a mini-batch and it is hard to apply to recurrent neural networks.

3 Layer normalization

We now consider the layer normalization method which is designed to overcome the drawbacks of batch normalization.

Notice that changes in the output of one layer will tend to cause highly correlated changes in the summed inputs to the next layer, especially with ReLU units whose outputs can change by a lot. This suggests the "covariate shift" problem can be reduced by fixing the mean and the variance of the summed inputs within each layer. We, thus, compute the layer normalization statistics over all the hidden units in the same layer as follows:

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l \qquad \sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2} \tag{3}$$

where H denotes the number of hidden units in a layer. The difference between Eq. (2) and Eq. (3) is that under layer normalization, all the hidden units in a layer share the same normalization terms μ and σ, but different training cases have different normalization terms. Unlike batch normalization, layer normaliztion does not impose any constraint on the size of a mini-batch and it can be used in the pure online regime with batch size 1.

批量归一化同样不适用于在线学习场景，也不适合那些因数据规模过大而不得不采用极小批量数据的分布式模型。

本文介绍了层归一化这一简单的归一化方法，旨在提升各类神经网络模型的训练速度。与批量归一化不同，该方法直接从隐藏层中神经元接收到的输入总和来估算归一化统计量，因此不会在训练样本之间引入任何新的依赖关系。研究结果表明，层归一化对循环神经网络效果显著，能够同时缩短多种现有循环神经网络模型的训练时间并提升其泛化性能。

2 背景

前馈神经网络是一种将输入模式$\mathbf{x}$ 映射为输出向量y的非线性映射函数。考虑深度前馈神经网络中的 l^{th} 隐藏层，设 a^l 为该层中神经元所有输入值求和后的向量表示。这些求和后的输入值是通过权重矩阵 W^l 进行线性投影，并结合自下而上的输入 h^l 计算得出的，具体计算方式如下：

$$a_i^l = {w_i^l}^\top h^l \qquad h_i^{l+1} = f(a_i^l + b_i^l) \tag{1}$$

其中 $f(\cdot)$为逐元素非线性函数，w_i^l 是输入到 i^{th} 隐藏单元的权重，而 b_i^l 则为标量偏置参数。神经网络中的这些参数是通过基于梯度的优化算法来学习的，其中的梯度则是通过反向传播计算得出的。

深度学习面临的一个挑战是，某一层中权重对应的梯度会高度依赖于前一层的神经元输出，尤其是当这些输出以高度相关的形式变化时。为此人们提出了批量归一化 [Ioffe和Szegedy, 2015] ，旨在减少这种不良的“协变量偏移”现象。该方法会对训练样本中每个隐藏单元的所有输入数据进行归一化处理。具体而言，对于 l^{th} 层中的 i^{th} 汇总后的输入数据，批量归一化方法会根据其在数据分布下的方差来重新调整这些输入数据的数值。

$$\bar{a}_i^l = \frac{g_i^l}{\sigma_i^l}\left(a_i^l - \mu_i^l\right) \qquad \mu_i^l = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[a_i^l\right] \qquad \sigma_i^l = \sqrt{\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\left(a_i^l - \mu_i^l\right)^2\right]} \tag{2}$$

其中 $\bar{a}_i^l$ 表示 l^{th} 层中 i^{th} 隐藏单元的归一化后的输入总和，而 g_i 则是用于在非线性激活函数之前对归一化后的激活值进行缩放的增益参数。需要注意的是，该期望值是在整个训练数据分布下计算的。通常而言，要精确计算方程(2)中的期望值是不切实际的，因为这需要使用当前的权重对整个训练数据集进行前向传播。因此，μ 和 σ 是通过当前小批量数据中的经验样本来估算的。这就对小批量数据的大小提出了限制，同时也使得该方法难以应用于循环神经网络。

3 层归一化

接下来我们将探讨层归一化方法，该方法旨在克服批量归一化的种种缺陷。

需要注意的是，某一层的输出发生变化往往会导致下一层输入总和出现高度相关的变化，尤其是对于那些输出波动较大的ReLU隐藏单元而言。这表明，通过固定每一层内输入总和的均值与方差，可以有效缓解“协变量偏移”问题。因此，我们按照如下方式计算同一层所有隐藏单元的层归一化统计量：

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l \qquad \sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2} \tag{3}$$

其中 H 表示某一层中的隐藏单元数量。方程(2)与方程(3)的区别在于：在层归一化的作用下，同一层中的所有隐藏单元会共享相同的归一化参数μ 和 σ，而不同的训练样本则拥有各自的归一化参数。与批量归一化不同，层归一化不对小批量数据的大小施加任何限制，因此即便在批量大小为1的纯在线学习场景中也能使用。

3.1 Layer normalized recurrent neural networks

The recent sequence to sequence models [Sutskever et al., 2014] utilize compact recurrent neural networks to solve sequential prediction problems in natural language processing. It is common among the NLP tasks to have different sentence lengths for different training cases. This is easy to deal with in an RNN because the same weights are used at every time-step. But when we apply batch normalization to an RNN in the obvious way, we need to to compute and store separate statistics for each time step in a sequence. This is problematic if a test sequence is longer than any of the training sequences. Layer normalization does not have such problem because its normalization terms depend only on the summed inputs to a layer at the current time-step. It also has only one set of gain and bias parameters shared over all time-steps.

In a standard RNN, the summed inputs in the recurrent layer are computed from the current input $\mathbf{x}^t$ and previous vector of hidden states $\mathbf{h}^{t-1}$ which are computed as $\mathbf{a}^t = W_{hh}h^{t-1} + W_{xh}\mathbf{x}^t$. The layer normalized recurrent layer re-centers and re-scales its activations using the extra normalization terms similar to Eq. (3):

$$\mathbf{h}^t = f\left[\frac{\mathbf{g}}{\sigma^t} \odot \left(\mathbf{a}^t - \mu^t\right) + \mathbf{b}\right] \qquad \mu^t = \frac{1}{H}\sum_{i=1}^{H} a_i^t \qquad \sigma^t = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^t - \mu^t\right)^2} \tag{4}$$

where W_{hh} is the recurrent hidden to hidden weights and W_{xh} are the bottom up input to hidden weights. $\odot$ is the element-wise multiplication between two vectors. $\mathbf{b}$ and $\mathbf{g}$ are defined as the bias and gain parameters of the same dimension as $\mathbf{h}^t$.

In a standard RNN, there is a tendency for the average magnitude of the summed inputs to the recurrent units to either grow or shrink at every time-step, leading to exploding or vanishing gradients. In a layer normalized RNN, the normalization terms make it invariant to re-scaling all of the summed inputs to a layer, which results in much more stable hidden-to-hidden dynamics.

4 Related work

Batch normalization has been previously extended to recurrent neural networks [Laurent et al., 2015, Amodei et al., 2015, Cooijmans et al., 2016]. The previous work [Cooijmans et al., 2016] suggests the best performance of recurrent batch normalization is obtained by keeping independent normalization statistics for each time-step. The authors show that initializing the gain parameter in the recurrent batch normalization layer to 0.1 makes significant difference in the final performance of the model. Our work is also related to weight normalization [Salimans and Kingma, 2016]. In weight normalization, instead of the variance, the L2 norm of the incoming weights is used to normalize the summed inputs to a neuron. Applying either weight normalization or batch normalization using expected statistics is equivalent to have a different parameterization of the original feed-forward neural network. Re-parameterization in the ReLU network was studied in the Path-normalized SGD [Neyshabur et al., 2015]. Our proposed layer normalization method, however, is not a re-parameterization of the original neural network. The layer normalized model, thus, has different invariance properties than the other methods, that we will study in the following section.

5 Analysis

In this section, we investigate the invariance properties of different normalization schemes.

5.1 Invariance under weights and data transformations

The proposed layer normalization is related to batch normalization and weight normalization. Although, their normalization scalars are computed differently, these methods can be summarized as normalizing the summed inputs a_i to a neuron through the two scalars μ and σ. They also learn an adaptive bias b and gain g for each neuron after the normalization.

$$h_i = f(\frac{g_i}{\sigma_i}(a_i - \mu_i) + b_i) \tag{5}$$

Note that for layer normalization and batch normalization, μ and σ is computed according to Eq. 2 and 3. In weight normalization, μ is 0, and $\sigma = \|w\|_2$.

3.1 经层归一化的循环神经网络

近期的序列到序列模型 [Sutskever等人 2014]采用结构紧凑的循环神经网络来解决自然语言处理中的序列预测问题。在各类NLP任务中，不同的训练样本往往具有不同的句子长度，这在循环神经网络中很容易处理，因为每个时间步都会使用相同的权重。然而，如果我们以常规方式对循环神经网络应用批量归一化，就需要为序列中的每个时间步分别计算并存储统计信息。一旦测试序列的长度超过任何训练序列的长度，就会出现问题。而层归一化则不存在此类问题，因为其归一化系数仅取决于当前时间步该层所有输入值的总和，且它也只有一组在所有时间步中共享的增益和偏置参数。

在标准的循环神经网络中，递归层中的输入总和是通过当前输入$\mathbf{x}^t$ 以及先前的隐藏状态向量$\mathbf{h}^{t-1}$ 计算得出的，而这些隐藏状态向量又是通过$\mathbf{a}^t = W_{hh}h^{t-1} + W_{xh}\mathbf{x}^t$计算得到的。层归一化递归层则利用类似于公式(3)的额外归一化项，对其激活值进行重新定位与重新缩放。

$$\mathbf{h}^t = f\left[\frac{\mathbf{g}}{\sigma^t} \odot \left(\mathbf{a}^t - \mu^t\right) + \mathbf{b}\right] \qquad \mu^t = \frac{1}{H}\sum_{i=1}^{H} a_i^t \qquad \sigma^t = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^t - \mu^t\right)^2} \tag{4}$$

其中 W_{hh} 表示循环隐藏层之间的权重， W_{xh} 则表示自下而上的输入数据与隐藏层之间的权重。$\odot$ 表示两个向量之间的逐元素乘法。$\mathbf{b}$ 和$\mathbf{g}$ 被定义为与$\mathbf{h}^t$具有相同维度的偏置参数和增益参数。

在标准的循环神经网络中，传递给各循环单元的输入数据之和的平均幅值往往会在每个时间步上不断增大或减小，从而导致梯度出现爆炸或消失的现象。而在经过层归一化的循环神经网络中，归一化机制使得该网络对将某一层的所有输入数据重新缩放这一操作具备不变性，进而让隐藏状态之间的动态变化更加稳定。

4 相关工作

Batch normalization has been previously extended to recurrent neural networks [Laurent et al., 2015, Amodei et al., 2015, Cooijmans et al., 2016]. The previous work [Cooijmans et al., 2016] suggests the best performance of recurrent batch normalization is obtained by keeping independent normalization statistics for each time-step. The authors show that initializing the gain parameter in the recurrent batch normalization layer to 0.1 makes significant difference in the final performance of the model. Our work is also related to weight normalization [Salimans and Kingma, 2016]. In weight normalization, instead of the variance, the L2 norm of the incoming weights is used to normalize the summed inputs to a neuron. Applying either weight normalization or batch normalization using expected statistics is equivalent to have a different parameterization of the original feed-forward neural network. Re-parameterization in the ReLU network was studied in the Path-normalized SGD [Neyshabur et al., 2015]. Our proposed layer normalization method, however, is not a re-parameterization of the original neural network. The layer normalized model, thus, has different invariance properties than the other methods, that we will study in the following section.

5 分析

在本节中，我们将研究不同归一化方案所具备的不变性特性。

5.1 权重与数据变换下的不变性

所提出的层归一化方法与批量归一化以及权重归一化存在关联。尽管这两种方法的归一化标量计算方式不同，但本质上都是通过 μ 和 σ 这两个标量来对传入神经元的输入数据 a_i 进行归一化处理。此外，这类方法还会为每个神经元学习一个自适应偏置 b 以及归一化后的增益值 g。

$$h_i = f(\frac{g_i}{\sigma_i}(a_i - \mu_i) + b_i) \tag{5}$$

需要注意的是，对于层归一化与批量归一化， μ 和 σ 是依据公式2和公式3计算得出的。而在权重归一化中， μ 的值为0， $\sigma = \|w\|_2$则另行确定。

	Weight matrix re-scaling	Weight matrix re-centering	Weight vector re-scaling	Dataset re-scaling	Dataset re-centering	Single training case re-scaling
Batch norm	Invariant	No	Invariant	Invariant	Invariant	No
Weight norm	Invariant	No	Invariant	No	No	No
Layer norm	Invariant	Invariant	No	Invariant	No	Invariant

Table 1: Invariance properties under the normalization methods.

Table 1 highlights the following invariance results for three normalization methods.

Weight re-scaling and re-centering: First, observe that under batch normalization and weight normalization, any re-scaling to the incoming weights w_i of a single neuron has no effect on the normalized summed inputs to a neuron. To be precise, under batch and weight normalization, if the weight vector is scaled by δ, the two scalar μ and σ will also be scaled by δ. The normalized summed inputs stays the same before and after scaling. So the batch and weight normalization are invariant to the re-scaling of the weights. Layer normalization, on the other hand, is not invariant to the individual scaling of the single weight vectors. Instead, layer normalization is invariant to scaling of the entire weight matrix and invariant to a shift to all of the incoming weights in the weight matrix. Let there be two sets of model parameters θ, θ' whose weight matrices W and W' differ by a scaling factor δ and all of the incoming weights in W' are also shifted by a constant vector $\boldsymbol{\gamma}$, that is $W' = \delta W + \mathbf{1}\boldsymbol{\gamma}^\top$. Under layer normalization, the two models effectively compute the same output:

$$\begin{aligned}\mathbf{h}' =& f(\frac{\mathbf{g}}{\sigma'}\left(W'\mathbf{x} - \mu'\right) + \mathbf{b}) = f(\frac{\mathbf{g}}{\sigma'}\left((\delta W + \mathbf{1}\boldsymbol{\gamma}^\top)\mathbf{x} - \mu'\right) + \mathbf{b}) \\ =& f(\frac{\mathbf{g}}{\sigma}\left(W\mathbf{x} - \mu\right) + \mathbf{b}) = \mathbf{h}. \end{aligned} \tag{6}$$

Notice that if normalization is only applied to the input before the weights, the model will not be invariant to re-scaling and re-centering of the weights.

Data re-scaling and re-centering: We can show that all the normalization methods are invariant to re-scaling the dataset by verifying that the summed inputs of neurons stays constant under the changes. Furthermore, layer normalization is invariant to re-scaling of individual training cases, because the normalization scalars μ and σ in Eq. (3) only depend on the current input data. Let $\mathbf{x}'$ be a new data point obtained by re-scaling $\mathbf{x}$ by δ. Then we have,

$$h'_i = f(\frac{g_i}{\sigma'}\left(w_i^\top \mathbf{x}' - \mu'\right) + b_i) = f(\frac{g_i}{\delta\sigma}\left(\delta w_i^\top \mathbf{x} - \delta\mu\right) + b_i) = h_i. \tag{7}$$

It is easy to see re-scaling individual data points does not change the model's prediction under layer normalization. Similar to the re-centering of the weight matrix in layer normalization, we can also show that batch normalization is invariant to re-centering of the dataset.

5.2 Geometry of parameter space during learning

We have investigated the invariance of the model's prediction under re-centering and re-scaling of the parameters. Learning, however, can behave very differently under different parameterizations, even though the models express the same underlying function. In this section, we analyze learning behavior through the geometry and the manifold of the parameter space. We show that the normalization scalar σ can implicitly reduce learning rate and makes learning more stable.

5.2.1 Riemannian metric

The learnable parameters in a statistical model form a smooth manifold that consists of all possible input-output relations of the model. For models whose output is a probability distribution, a natural way to measure the separation of two points on this manifold is the Kullback-Leibler divergence between their model output distributions. Under the KL divergence metric, the parameter space is a Riemannian manifold.

The curvature of a Riemannian manifold is entirely captured by its Riemannian metric, whose quadratic form is denoted as ds^2. That is the infinitesimal distance in the tangent space at a point in the parameter space. Intuitively, it measures the changes in the model output from the parameter space along a tangent direction. The Riemannian metric under KL was previously studied [Amari, 1998] and was shown to be well approximated under second order Taylor expansion using the Fisher

	权重矩阵 重新缩放	权重矩阵 重新居中	权重向量 重新缩放	数据集 重新缩放	数据集 重新居中	单个训练样本 重新缩放
批量归一化	不变性	No	不变性	不变性	不变性	No
权重归一化	不变性	No	不变性	No	No	No
层归一化	不变性	不变性	No	不变性	No	不变性

表1：各归一化方法下的不变性特性。

表1展示了三种归一化方法的以下不变性测试结果。

Weight re-scaling and re-centering: First, observe that under batch normalization and weight normalization, any re-scaling to the incoming weights w_i of a single neuron has no effect on the normalized summed inputs to a neuron. To be precise, under batch and weight normalization, if the weight vector is scaled by δ, the two scalar μ and σ will also be scaled by δ. The normalized summed inputs stays the same before and after scaling. So the batch and weight normalization are invariant to the re-scaling of the weights. Layer normalization, on the other hand, is not invariant to the individual scaling of the single weight vectors. Instead, layer normalization is invariant to scaling of the entire weight matrix and invariant to a shift to all of the incoming weights in the weight matrix. Let there be two sets of model parameters θ, θ' whose weight matrices W and W' differ by a scaling factor δ and all of the incoming weights in W' are also shifted by a constant vector $\boldsymbol{\gamma}$, that is $W' = \delta W + \mathbf{1}\boldsymbol{\gamma}^\top$. Under layer normalization, the two models effectively compute the same output:

$$\begin{aligned}\mathbf{h}' =& f(\frac{\mathbf{g}}{\sigma'}\left(W'\mathbf{x} - \mu'\right) + \mathbf{b}) = f(\frac{\mathbf{g}}{\sigma'}\left((\delta W + \mathbf{1}\boldsymbol{\gamma}^\top)\mathbf{x} - \mu'\right) + \mathbf{b}) \\ =& f(\frac{\mathbf{g}}{\sigma}\left(W\mathbf{x} - \mu\right) + \mathbf{b}) = \mathbf{h}. \end{aligned} \tag{6}$$

请注意，如果仅对权重之前的输入数据应用归一化处理，该模型将不具备对权重重新缩放及重新居中的不变性。

数据的重缩放与重新居中：我们可以通过验证神经元输入数据的总和在变化后依然保持不变，从而证明所有的归一化方法都对数据集的重缩放具有不变性。此外，层归一化也对单个训练样本的重缩放具备不变性，因为公式(3)中的归一化标量 μ和 σ 仅取决于当前的输入数据。设$\mathbf{x}'$是通过将$\mathbf{x}$ 乘以 δ进行重缩放后得到的新数据点，那么我们有：

$$h'_i = f(\frac{g_i}{\sigma'}\left(w_i^\top \mathbf{x}' - \mu'\right) + b_i) = f(\frac{g_i}{\delta\sigma}\left(\delta w_i^\top \mathbf{x} - \delta\mu\right) + b_i) = h_i. \tag{7}$$

不难看出，在层归一化机制下，对单个数据点进行重新缩放并不会改变模型的预测结果。与层归一化中对权重矩阵进行重新居中的情况类似，我们同样可以证明批量归一化对于数据集的重新居中操作也具备不变性。

5.2 学习过程中参数空间的几何结构

我们研究了在参数的重新居中与重新缩放作用下，模型预测结果的不变性。然而，即便这些模型表达的是相同的底层函数，其学习过程在不同参数化设置下仍可能表现出截然不同的行为。在本节中，我们将通过参数空间的几何结构与流形来分析学习行为。研究表明，归一化标量 σ 能够隐式地降低学习率，进而提升学习的稳定性。

5.2.1 黎曼度量

在统计模型中，那些可学习参数构成了一种平滑的流形，该流形包含了该模型所有可能的输入输出关系。对于那些输出为概率分布的模型而言，衡量该流形上两点之间分离程度的自然方式，就是计算它们对应模型输出概率分布之间的库尔贝克-莱布勒散度。在KL散度度量的框架下，参数空间便成了一种黎曼流形。

黎曼流形的曲率完全由其黎曼度量决定，而该度量的二次型则用 ds^2表示。它实际上就是参数空间中某一点处切空间内的无穷小距离。从直观上讲，它用于衡量模型输出在参数空间沿切线方向的变化程度。此前已有研究 [Amari,1998] 探讨过在KL散度框架下的黎曼度量，结果表明利用费希尔方法通过二阶泰勒展开即可对其实现很好的近似。

information matrix:

$$ds^2 = \mathrm{D}_{\mathrm{KL}}\big[P(y\,|\,\mathbf{x};\,\theta)\|P(y\,|\,\mathbf{x};\,\theta+\delta)\big] \approx \frac{1}{2}\delta^\top F(\theta)\delta, \tag{8}$$

$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x}),y\sim P(y\,|\,\mathbf{x})}\left[\frac{\partial \log P(y\,|\,\mathbf{x};\,\theta)}{\partial\theta}\frac{\partial \log P(y\,|\,\mathbf{x};\,\theta)}{\partial\theta}^\top\right], \tag{9}$$

where, δ is a small change to the parameters. The Riemannian metric above presents a geometric view of parameter spaces. The following analysis of the Riemannian metric provides some insight into how normalization methods could help in training neural networks.

5.2.2 The geometry of normalized generalized linear models

We focus our geometric analysis on the generalized linear model. The results from the following analysis can be easily applied to understand deep neural networks with block-diagonal approximation to the Fisher information matrix, where each block corresponds to the parameters for a single neuron.

A generalized linear model (GLM) can be regarded as parameterizing an output distribution from the exponential family using a weight vector w and bias scalar b. To be consistent with the previous sections, the log likelihood of the GLM can be written using the summed inputs a as the following:

$$\log P(y\,|\,\mathbf{x};\,w,b) = \frac{(a+b)y-\eta(a+b)}{\phi} + c(y,\phi), \tag{10}$$

$$\mathbb{E}[y\,|\,\mathbf{x}] = f(a+b) = f(w^\top\mathbf{x}+b),\;\; \mathrm{Var}[y\,|\,\mathbf{x}] = \phi f'(a+b), \tag{11}$$

where, $f(\cdot)$ is the transfer function that is the analog of the non-linearity in neural networks, $f'(\cdot)$ is the derivative of the transfer function, $\eta(\cdot)$ is a real valued function and $c(\cdot)$ is the log partition function. ϕ is a constant that scales the output variance. Assume a H-dimensional output vector $\mathbf{y} = [y_1, y_2, \cdots, y_H]$ is modeled using H independent GLMs and $\log P(\mathbf{y}\,|\,\mathbf{x};\,W,\mathbf{b}) = \sum_{i=1}^H \log P(y_i\,|\,\mathbf{x};\,w_i,b_i)$. Let W be the weight matrix whose rows are the weight vectors of the individual GLMs, $\mathbf{b}$ denote the bias vector of length H and $\mathrm{vec}(\cdot)$ denote the Kronecker vector operator. The Fisher information matrix for the multi-dimensional GLM with respect to its parameters $\theta = [w_1^\top, b_1, \cdots, w_H^\top, b_H]^\top = \mathrm{vec}([W,\mathbf{b}]^\top)$ is simply the expected Kronecker product of the data features and the output covariance matrix:

$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[\mathbf{y}\,|\,\mathbf{x}]}{\phi^2}\otimes\begin{bmatrix}\mathbf{x}\mathbf{x}^\top & \mathbf{x}\\ \mathbf{x}^\top & 1\end{bmatrix}\right]. \tag{12}$$

We obtain normalized GLMs by applying the normalization methods to the summed inputs a in the original model through μ and σ. Without loss of generality, we denote $\bar{F}$ as the Fisher information matrix under the normalized multi-dimensional GLM with the additional gain parameters $\theta = \mathrm{vec}([W,\mathbf{b},\mathbf{g}]^\top)$:

$$\bar{F}(\theta) = \begin{bmatrix}\bar{F}_{11} & \cdots & \bar{F}_{1H}\\ \vdots & \ddots & \vdots\\ \bar{F}_{H1} & \cdots & \bar{F}_{HH}\end{bmatrix},\quad \bar{F}_{ij} = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[y_i,\,y_j\,|\,\mathbf{x}]}{\phi^2}\begin{bmatrix}\frac{g_ig_j}{\sigma_i\sigma_j}\chi_i\chi_j^\top & \chi_i\frac{g_i}{\sigma_i} & \chi_i\frac{g_i(a_j-\mu_j)}{\sigma_i\sigma_j}\\ \chi_j^\top\frac{g_j}{\sigma_j} & 1 & \frac{a_j-\mu_j}{\sigma_j}\\ \chi_j^\top\frac{g_j(a_i-\mu_i)}{\sigma_i\sigma_j} & \frac{a_i-\mu_i}{\sigma_i} & \frac{(a_i-\mu_i)(a_j-\mu_j)}{\sigma_i\sigma_j}\end{bmatrix}\right] \tag{13}$$

$$\chi_i = \mathbf{x} - \frac{\partial\mu_i}{\partial w_i} - \frac{a_i-\mu_i}{\sigma_i}\frac{\partial\sigma_i}{\partial w_i}. \tag{14}$$

Implicit learning rate reduction through the growth of the weight vector: Notice that, comparing to standard GLM, the block $\bar{F}_{ij}$ along the weight vector w_i direction is scaled by the gain parameters and the normalization scalar σ_i. If the norm of the weight vector w_i grows twice as large, even though the model's output remains the same, the Fisher information matrix will be different. The curvature along the w_i direction will change by a factor of $\frac{1}{2}$ because the σ_i will also be twice as large. As a result, for the same parameter update in the normalized model, the norm of the weight vector effectively controls the learning rate for the weight vector. During learning, it is harder to change the orientation of the weight vector with large norm. The normalization methods, therefore,

信息矩阵：

$$ds^2 = \mathrm{D}_{\mathrm{KL}}\big[P(y\,|\,\mathbf{x};\,\theta)\|P(y\,|\,\mathbf{x};\,\theta+\delta)\big] \approx \frac{1}{2}\delta^\top F(\theta)\delta, \tag{8}$$

$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x}),y\sim P(y\,|\,\mathbf{x})}\left[\frac{\partial \log P(y\,|\,\mathbf{x};\,\theta)}{\partial\theta}\frac{\partial \log P(y\,|\,\mathbf{x};\,\theta)}{\partial\theta}^\top\right], \tag{9}$$

其中， δ 表示参数的微小变化。上述的黎曼度量从几何角度展示了参数空间。对黎曼度量的进一步分析有助于我们理解归一化方法在训练神经网络时能发挥怎样的作用。

5.2.2 规范化广义线性模型的几何结构

我们的几何分析重点放在广义线性模型上。通过以下分析得出的结果可被轻松应用于理解那些对费希尔信息矩阵采用块对角线近似结构的深度神经网络，其中每个块对应一个单个神经元的参数。

广义线性模型（GLM模型）可被视为利用权重向量 w 和偏置标量 b，从指数族中确定输出分布的参数形式。为与前几节保持一致，该GLM模型的对数似然可通过将所有输入数据 a 相加后按如下方式表示：

$$\log P(y\,|\,\mathbf{x};\,w,b) = \frac{(a+b)y-\eta(a+b)}{\phi} + c(y,\phi), \tag{10}$$

$$\mathbb{E}[y\,|\,\mathbf{x}] = f(a+b) = f(w^\top\mathbf{x}+b),\;\; \mathrm{Var}[y\,|\,\mathbf{x}] = \phi f'(a+b), \tag{11}$$

where, $f(\cdot)$ is the transfer function that is the analog of the non-linearity in neural networks, $f'(\cdot)$ is the derivative of the transfer function, $\eta(\cdot)$ is a real valued function and $c(\cdot)$ is the log partition function. ϕ is a constant that scales the output variance. Assume a H-dimensional output vector $\mathbf{y} = [y_1, y_2, \cdots, y_H]$ is modeled using H independent GLMs and $\log P(\mathbf{y}\,|\,\mathbf{x};\,W,\mathbf{b}) = \sum_{i=1}^H \log P(y_i\,|\,\mathbf{x};\,w_i,b_i)$. Let W be the weight matrix whose rows are the weight vectors of the individual GLMs, $\mathbf{b}$ denote the bias vector of length H and $\mathrm{vec}(\cdot)$ denote the Kronecker vector operator. The Fisher information matrix for the multi-dimensional GLM with respect to its parameters $\theta = [w_1^\top, b_1, \cdots, w_H^\top, b_H]^\top = \mathrm{vec}([W,\mathbf{b}]^\top)$ is simply the expected Kronecker product of the data features and the output covariance matrix:

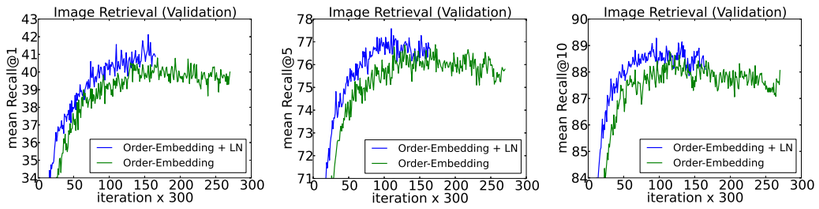
$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[\mathbf{y}\,|\,\mathbf{x}]}{\phi^2}\otimes\begin{bmatrix}\mathbf{x}\mathbf{x}^\top & \mathbf{x}\\ \mathbf{x}^\top & 1\end{bmatrix}\right]. \tag{12}$$

我们通过将标准化方法应用于原始模型中的 a 输入数据，并借助 μ 与 σ，从而得到标准化的GLM模型。为简化表述，在包含额外增益参数$\theta = \mathrm{vec}([W,\mathbf{b}、\mathbf{g}]^\top)$的标准多维GLM下，我们将 F 称为费希尔信息矩阵。

$$\bar{F}(\theta) = \begin{bmatrix}\bar{F}_{11} & \cdots & \bar{F}_{1H}\\ \vdots & \ddots & \vdots\\ \bar{F}_{H1} & \cdots & \bar{F}_{HH}\end{bmatrix},\quad \bar{F}_{ij} = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[y_i,\,y_j\,|\,\mathbf{x}]}{\phi^2}\begin{bmatrix}\frac{g_ig_j}{\sigma_i\sigma_j}\chi_i\chi_j^\top & \chi_i\frac{g_i}{\sigma_i} & \chi_i\frac{g_i(a_j-\mu_j)}{\sigma_i\sigma_j}\\ \chi_j^\top\frac{g_j}{\sigma_j} & 1 & \frac{a_j-\mu_j}{\sigma_j}\\ \chi_j^\top\frac{g_j(a_i-\mu_i)}{\sigma_i\sigma_j} & \frac{a_i-\mu_i}{\sigma_i} & \frac{(a_i-\mu_i)(a_j-\mu_j)}{\sigma_i\sigma_j}\end{bmatrix}\right] \tag{13}$$

$$\chi_i = \mathbf{x} - \frac{\partial\mu_i}{\partial w_i} - \frac{a_i-\mu_i}{\sigma_i}\frac{\partial\sigma_i}{\partial w_i}. \tag{14}$$

通过权重向量的增长实现隐式的学习率降低：请注意，与标准GLM模型相比，沿权重向量 w_i 方向的数值会经过增益参数以及归一化标量 σ_i的缩放处理。即便模型的输出结果保持不变，但如果权重向量 w_i 的范数增大一倍，费希尔信息矩阵也会随之改变。由于 σ_i 的值同样会变为两倍，沿 w_i 方向的曲率将会以1/2的比例发生变化。因此，在经过归一化的模型中，对于相同的参数更新而言，权重向量的范数实际上起到了控制其学习率的作用。在学习过程中，当权重向量的范数较大时，要改变其方向就会更加困难。正因如此，这些归一化方法会对权重向量产生隐式的“提前停止”效应，有助于稳定学习过程并使其趋向收敛。



(a) Recall@1 (b) Recall@5 (c) Recall@10

Figure 1: Recall@K curves using order-embeddings with and without layer normalization.

Model	MSCOCO							
	Caption Retrieval				Image Retrieval			
	R@1	R@5	R@10	Mean r	R@1	R@5	R@10	Mean r
Sym [Vendrov et al., 2016]	45.4		88.7	5.8	36.3		85.8	9.0
OE [Vendrov et al., 2016]	46.7		88.9	5.7	37.9		85.9	8.1
OE (ours)	46.6	79.3	89.1	5.2	37.8	73.6	85.7	7.9
OE + LN	**48.5**	**80.6**	**89.8**	**5.1**	**38.9**	**74.3**	**86.3**	**7.6**

Table 2: Average results across 5 test splits for caption and image retrieval. **R@K** is Recall@K (high is good). **Mean** r is the mean rank (low is good). Sym corresponds to the symmetric baseline while OE indicates order-embeddings.

have an implicit "early stopping" effect on the weight vectors and help to stabilize learning towards convergence.

Learning the magnitude of incoming weights: In normalized models, the magnitude of the incoming weights is explicitly parameterized by the gain parameters. We compare how the model output changes between updating the gain parameters in the normalized GLM and updating the magnitude of the equivalent weights under original parameterization during learning. The direction along the gain parameters in F captures the geometry for the magnitude of the incoming weights. We show that Riemannian metric along the magnitude of the incoming weights for the standard GLM is scaled by the norm of its input, whereas learning the gain parameters for the batch normalized and layer normalized models depends only on the magnitude of the prediction error. Learning the magnitude of incoming weights in the normalized model is therefore, more robust to the scaling of the input and its parameters than in the standard model. See Appendix for detailed derivations.

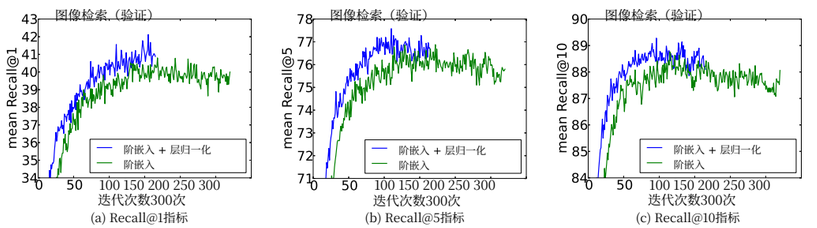
6 Experimental results

We perform experiments with layer normalization on 6 tasks, with a focus on recurrent neural networks: image-sentence ranking, question-answering, contextual language modelling, generative modelling, handwriting sequence generation and MNIST classification. Unless otherwise noted, the default initialization of layer normalization is to set the adaptive gains to 1 and the biases to 0 in the experiments.

6.1 Order embeddings of images and language

In this experiment, we apply layer normalization to the recently proposed order-embeddings model of Vendrov et al. [2016] for learning a joint embedding space of images and sentences. We follow the same experimental protocol as Vendrov et al. [2016] and modify their publicly available code to incorporate layer normalization [1] which utilizes Theano [Team et al., 2016]. Images and sentences from the Microsoft COCO dataset [Lin et al., 2014] are embedded into a common vector space, where a GRU [Cho et al., 2014] is used to encode sentences and the outputs of a pre-trained VGG ConvNet [Simonyan and Zisserman, 2015] (10-crop) are used to encode images. The order-embedding model represents images and sentences as a 2-level partial ordering and replaces the cosine similarity scoring function used in Kiros et al. [2014] with an asymmetric one.

[1] `https://github.com/ivendrov/order-embedding`



(a) Recall@1指标 (b) Recall@5指标 (c) Recall@10指标

图1：采用不同阶嵌入结构且在有无层归一化情况下的Recall@K曲线。

模型	MSCOCO数据集							
	标题检索				图像检索			
	R@1	R@5	R@10	均值 r	R@1	R@5	R@10	均值 r
对称基线方法 [Vendrov等人，2016]	45.4		88.7	5.8	36.3		85.8	9.0
顺序嵌入方法 [Vendrov等人，2016]	46.7		88.9	5.7	37.9		85.9	8.1
顺序嵌入方法（我们的方案）	46.6	79.3	89.1	5.2	37.8	73.6	85.7	7.9
顺序嵌入方法 + 层归一化	**48.5**	**80.6**	**89.8**	**5.1**	**38.9**	**74.3**	**86.3**	**7.6**

表2：标题检索与图像检索在5组测试数据划分下的平均结果。**R@K** 即Recall@K（数值越高表示性能越好）。**Mean** r 为平均排名（数值越低表示性能越好）。Sym代表对称基线方法，而OE则表示阶嵌入结构。

从而对权重向量起到隐式的‘提前停止’作用，助力让学习过程更加稳定并最终达到收敛状态。

学习输入权重的幅值：在经过归一化的模型中，输入权重的幅值是通过增益参数来明确指定的。我们比较了在归一化GLM模型中更新增益参数，与在学习过程中按原始参数化方式更新等效权重的幅值时，模型输出的变化情况。F 中沿¯增益参数的方向决定了输入权重幅值的几何结构。研究表明，标准GLM模型中沿输入权重幅值的黎曼度量会由其输入数据的范数进行缩放，而批量归一化模型和层归一化模型中的增益参数学习则仅取决于预测误差的幅值。因此，归一化模型中学习输入权重的幅值对于输入数据及其参数的缩放更具鲁棒性。详细推导见附录。

6 实验结果

我们在6项任务上采用了层归一化技术进行实验，重点研究的是循环神经网络应用场景，包括图像-句子排序、问答系统、上下文语言建模、生成建模、手写序列生成以及MNIST数据集分类。除非另有说明，这些实验中层归一化的默认初始化方式为将自适应增益设置为1，偏置值设置为0。

6.1 图像与语言的顺序嵌入

In this experiment, we apply layer normalization to the recently proposed order-embeddings model of Vendrov et al. [2016] for learning a joint embedding space of images and sentences. We follow the same experimental protocol as Vendrov et al. [2016] and modify their publicly available code to incorporate layer normalization [1] which utilizes Theano [Team et al., 2016]. Images and sentences from the Microsoft COCO dataset [Lin et al., 2014] are embedded into a common vector space, where a GRU [Cho et al., 2014] is used to encode sentences and the outputs of a pre-trained VGG ConvNet [Simonyan and Zisserman, 2015] (10-crop) are used to encode images. The order-embedding model represents images and sentences as a 2-level partial ordering and replaces the cosine similarity scoring function used in Kiros et al. [2014] with an asymmetric one.

[1] https://github.com/ivendrov/order-embeddi ng

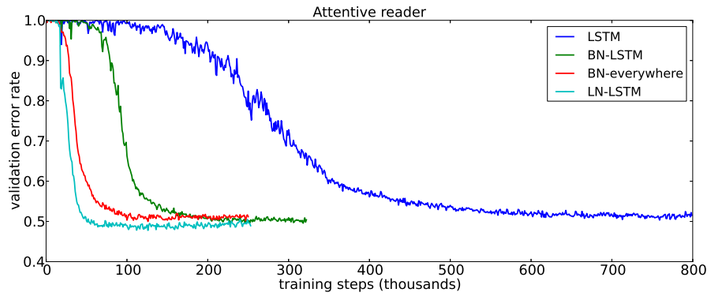


Figure 2: Validation curves for the attentive reader model. BN results are taken from [Cooijmans et al., 2016].

We trained two models: the baseline order-embedding model as well as the same model with layer normalization applied to the GRU. After every 300 iterations, we compute Recall@K (R@K) values on a held out validation set and save the model whenever R@K improves. The best performing models are then evaluated on 5 separate test sets, each containing 1000 images and 5000 captions, for which the mean results are reported. Both models use Adam [Kingma and Ba, 2014] with the same initial hyperparameters and both models are trained using the same architectural choices as used in Vendrov et al. [2016]. We refer the reader to the appendix for a description of how layer normalization is applied to GRU.

Figure 1 illustrates the validation curves of the models, with and without layer normalization. We plot R@1, R@5 and R@10 for the image retrieval task. We observe that layer normalization offers a per-iteration speedup across all metrics and converges to its best validation model in 60% of the time it takes the baseline model to do so. In Table 2, the test set results are reported from which we observe that layer normalization also results in improved generalization over the original model. The results we report are state-of-the-art for RNN embedding models, with only the structure-preserving model of Wang et al. [2016] reporting better results on this task. However, they evaluate under different conditions (1 test set instead of the mean over 5) and are thus not directly comparable.

6.2 Teaching machines to read and comprehend

In order to compare layer normalization to the recently proposed recurrent batch normalization [Cooijmans et al., 2016], we train an unidirectional attentive reader model on the CNN corpus both introduced by Hermann et al. [2015]. This is a question-answering task where a query description about a passage must be answered by filling in a blank. The data is anonymized such that entities are given randomized tokens to prevent degenerate solutions, which are consistently permuted during training and evaluation. We follow the same experimental protocol as Cooijmans et al. [2016] and modify their public code to incorporate layer normalization [2] which uses Theano [Team et al., 2016]. We obtained the pre-processed dataset used by Cooijmans et al. [2016] which differs from the original experiments of Hermann et al. [2015] in that each passage is limited to 4 sentences. In Cooijmans et al. [2016], two variants of recurrent batch normalization are used: one where BN is only applied to the LSTM while the other applies BN everywhere throughout the model. In our experiment, we only apply layer normalization within the LSTM.

The results of this experiment are shown in Figure 2. We observe that layer normalization not only trains faster but converges to a better validation result over both the baseline and BN variants. In Cooijmans et al. [2016], it is argued that the scale parameter in BN must be carefully chosen and is set to 0.1 in their experiments. We experimented with layer normalization for both 1.0 and 0.1 scale initialization and found that the former model performed significantly better. This demonstrates that layer normalization is not sensitive to the initial scale in the same way that recurrent BN is. [3]

6.3 Skip-thought vectors

Skip-thoughts [Kiros et al., 2015] is a generalization of the skip-gram model [Mikolov et al., 2013] for learning unsupervised distributed sentence representations. Given contiguous text, a sentence is

[2] `https://github.com/cooijmanstim/Attentive_reader/tree/bn`

[3] We only produce results on the validation set, as in the case of Cooijmans et al. [2016]

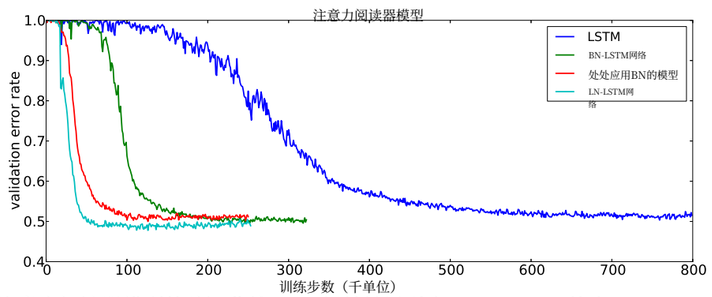


图2：注意力阅读器模型的验证曲线。BN相关结果引自 [Cooijmans等人， 2016]。

我们训练了两个模型：一个是作为基准模型的普通阶嵌入模型，另一个则是在GRU层上应用了层归一化的相同模型。每隔300次迭代，我们就会在预留的验证集上计算Recall@K（R@K）值，一旦R@K值有所提升就会保存该模型。表现最佳的模型随后会在5个独立的测试集上进行评估，每个测试集包含1000张图片和5000条标题检索结果，最终会汇报这些测试集的平均性能指标。这两个模型都采用Adam [Kingma和Ba 2014] 优化算法，并使用相同的初始超参数；同时，它们的架构设计也与Vendrov等人 [2016]的研究中所采用的方案一致。关于如何在GRU层上应用层归一化的详细说明，可参阅附录。

图1展示了应用了层归一化与未应用层归一化的模型的验证曲线。我们针对图像检索任务绘制了R@1、R@5和R@10指标。实验结果表明，层归一化在所有指标上都能提升每次迭代的处理速度，并且在达到最佳验证性能所需时间仅为基准模型的一半左右。表2列出了测试集的评估结果，从中可以看出层归一化同样提升了模型相对于原始模型的泛化能力。我们所报告的结果是目前循环神经网络嵌入模型中的最优水平，仅有Wang等人 [2016] 提出的保持结构不变的模型在该任务上取得了更好的成绩。不过，他们的评估条件有所不同（使用单个测试集而非5个测试集的平均值），因此两者无法直接比较。

6.2 教导机器进行阅读与理解

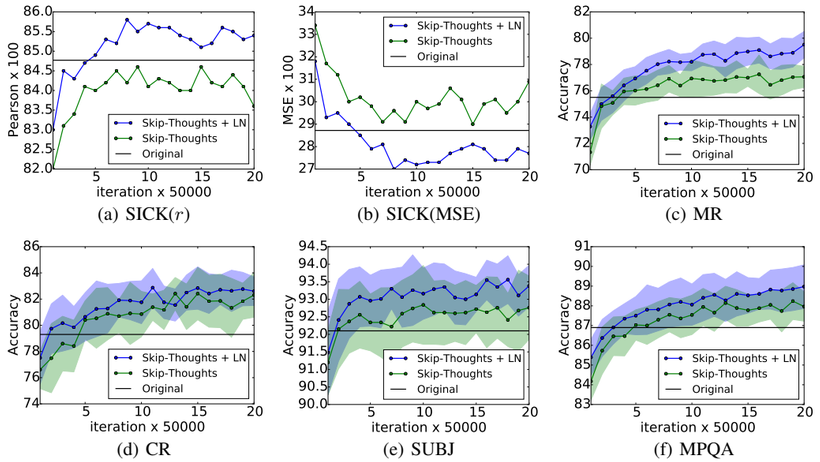
In order to compare layer normalization to the recently proposed recurrent batch normalization [Cooijmans et al., 2016], we train an unidirectional attentive reader model on the CNN corpus both introduced by Hermann et al. [2015]. This is a question-answering task where a query description about a passage must be answered by filling in a blank. The data is anonymized such that entities are given randomized tokens to prevent degenerate solutions, which are consistently permuted during training and evaluation. We follow the same experimental protocol as Cooijmans et al. [2016] and modify their public code to incorporate layer normalization [2] which uses Theano [Team et al., 2016]. We obtained the pre-processed dataset used by Cooijmans et al. [2016] which differs from the original experiments of Hermann et al. [2015] in that each passage is limited to 4 sentences. In Cooijmans et al. [2016], two variants of recurrent batch normalization are used: one where BN is only applied to the LSTM while the other applies BN everywhere throughout the model. In our experiment, we only apply layer normalization within the LSTM.

该实验的结果如图2所示。我们发现，与基准模型及各类BN变体相比，层归一化不仅训练速度更快，还能获得更好的验证结果。Cooijmans等人 [2016], 指出，BN中的缩放参数需要精心选取，在他们的实验中该参数被设置为0.1。我们分别尝试了初始缩放值为1.0和0.1的层归一化方案，结果发现前者性能明显更优。这表明层归一化对初始缩放值的敏感度并不如循环BN那么高。 [3]

6.3 跳跃思维向量

跳跃思维 [Kiros等人提出， 2015] 它是 [Mikolov等人开发的跳过词模型的一种泛化形式，旨在用于学习无监督的分布式句子表示。对于连续的文本而言，一个句子会被

[2] https://github.com/cooijmanstim/Attentive_reader/tree/bn [3] 与Cooijmans等人的做法一致，我们仅在验证集上展示实验结果。 [2016]



(a) SICK(r) (b) SICK(MSE) (c) MR

(d) CR (e) SUBJ (f) MPQA

Figure 3: Performance of skip-thought vectors with and without layer normalization on downstream tasks as a function of training iterations. The original lines are the reported results in [Kiros et al., 2015]. Plots with error use 10-fold cross validation. Best seen in color.

Method	**SICK**(r)	**SICK**(ρ)	**SICK**(MSE)	**MR**	**CR**	**SUBJ**	**MPQA**
Original [Kiros et al., 2015]	0.848	0.778	0.287	75.5	79.3	92.1	86.9
Ours	0.842	0.767	0.298	77.3	81.8	92.6	87.9
Ours + LN	0.854	0.785	0.277	**79.5**	82.6	93.4	89.0
Ours + LN †	**0.858**	**0.788**	**0.270**	79.4	**83.1**	**93.7**	**89.3**

Table 3: Skip-thoughts results. The first two evaluation columns indicate Pearson and Spearman correlation, the third is mean squared error and the remaining indicate classification accuracy. Higher is better for all evaluations except MSE. Our models were trained for 1M iterations with the exception of (†) which was trained for 1 month (approximately 1.7M iterations)

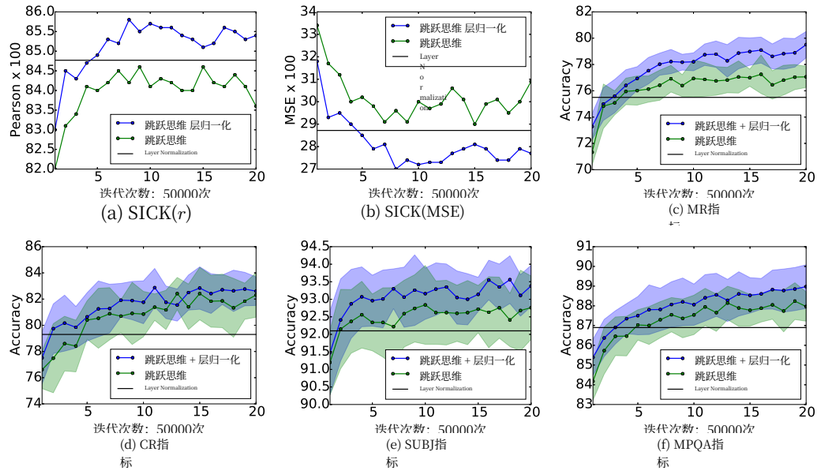
encoded with a encoder RNN and decoder RNNs are used to predict the surrounding sentences. Kiros et al. [2015] showed that this model could produce generic sentence representations that perform well on several tasks without being fine-tuned. However, training this model is time-consuming, requiring several days of training in order to produce meaningful results.

In this experiment we determine to what effect layer normalization can speed up training. Using the publicly available code of Kiros et al. [2015] [4], we train two models on the BookCorpus dataset [Zhu et al., 2015]: one with and one without layer normalization. These experiments are performed with Theano [Team et al., 2016]. We adhere to the experimental setup used in Kiros et al. [2015], training a 2400-dimensional sentence encoder with the same hyperparameters. Given the size of the states used, it is conceivable layer normalization would produce slower per-iteration updates than without. However, we found that provided CNMeM [5] is used, there was no significant difference between the two models. We checkpoint both models after every 50,000 iterations and evaluate their performance on five tasks: semantic-relatedness (SICK) [Marelli et al., 2014], movie review sentiment (MR) [Pang and Lee, 2005], customer product reviews (CR) [Hu and Liu, 2004], subjectivity/objectivity classification (SUBJ) [Pang and Lee, 2004] and opinion polarity (MPQA) [Wiebe et al., 2005]. We plot the performance of both models for each checkpoint on all tasks to determine whether the performance rate can be improved with LN.

The experimental results are illustrated in Figure 3. We observe that applying layer normalization results both in speedup over the baseline as well as better final results after 1M iterations are performed as shown in Table 3. We also let the model with layer normalization train for a total of a month, resulting in further performance gains across all but one task. We note that the performance

[4] https://github.com/ryankiros/skip-thoughts

[5] https://github.com/NVIDIA/cnmem



(a) SICK(r) (b) SICK(MSE) (c) MR指标

(d) CR指标 (e) SUBJ指标 (f) MPQA指标

图3：随着训练迭代次数的增加，带有层归一化与未带层归一化的跳过思维向量在下游任务上的性能表现。原始曲线为 [Kiros等人2015]报告的结果；带有误差标记的曲线则是通过10折交叉验证得出的结果，建议以彩色版本查看以获得最佳效果。

方法	**SICK**(r)	**SICK**(ρ)	**SICK**(MSE)	**MR**	**CR**	**SUBJ**	**MPQA**
原始方法：[Kiros等人，2015]	0.848	0.778	0.287	75.5	79.3	92.1	86.9
Ours	0.842	0.767	0.298	77.3	81.8	92.6	87.9
我们的方法：+层归一化	0.854	0.785	0.277	**79.5**	82.6	93.4	89.0
我们的方法：+层归一化†	**0.858**	**0.788**	**0.270**	79.4	**83.1**	**93.7**	**89.3**

表3：跳跃思维的实验结果。前两列评估指标分别代表皮尔逊相关系数和斯皮尔曼相关系数，第三列为均方误差，其余列则为分类准确率。除均方误差外，各项指标的值越高越好。我们的模型均经过了100万次迭代训练，只有(†)是经过1个月训练（约170万次迭代）。

通过编码器RNN对文本进行编码，再利用解码器RNNs来预测周围的句子。Kiros等人[2015]证明，该模型无需进一步微调即可在多项任务上生成性能优异的通用句子表示。不过，训练这种模型耗时较长，往往需要数天的训练时间才能获得有意义的结果。

In this experiment we determine to what effect layer normalization can speed up training. Using the publicly available code of Kiros et al. [2015] [4], we train two models on the BookCorpus dataset [Zhu et al., 2015]: one with and one without layer normalization. These experiments are performed with Theano [Team et al., 2016]. We adhere to the experimental setup used in Kiros et al. [2015], training a 2400-dimensional sentence encoder with the same hyperparameters. Given the size of the states used, it is conceivable layer normalization would produce slower per-iteration updates than without. However, we found that provided CNMeM [5] is used, there was no significant difference between the two models. We checkpoint both models after every 50,000 iterations and evaluate their performance on five tasks: semantic-relatedness (SICK) [Marelli et al., 2014], movie review sentiment (MR) [Pang and Lee, 2005], customer product reviews (CR) [Hu and Liu, 2004], subjectivity/objectivity classification (SUBJ) [Pang and Lee, 2004] and opinion polarity (MPQA) [Wiebe et al., 2005]. We plot the performance of both models for each checkpoint on all tasks to determine whether the performance rate can be improved with LN.

实验结果如图3所示。我们可以发现，应用层归一化不仅使得处理速度相较于基准模型有所提升，而且在完成100万次迭代后，最终效果也更为优异，具体数据见表3。此外，我们还让采用层归一化的模型训练了整整一个月，结果除了一项任务外，其余所有任务的性能都进一步得到提升。需要指出的是，这种性能提升

[4] https://github.com/ryankiros/skip-thoughts [5] https://github.com/NVIDIA/cnmem

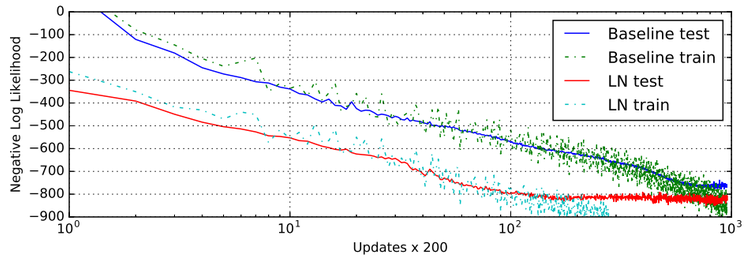


Figure 5: Handwriting sequence generation model negative log likelihood with and without layer normalization. The models are trained with mini-batch size of 8 and sequence length of 500,

differences between the original reported results and ours are likely due to the fact that the publicly available code does not condition at each timestep of the decoder, where the original model does.

6.4 Modeling binarized MNIST using DRAW

We also experimented with the generative modeling on the MNIST dataset. Deep Recurrent Attention Writer (DRAW) [Gregor et al., 2015] has previously achieved the state-of-the-art performance on modeling the distribution of MNIST digits. The model uses a differential attention mechanism and a recurrent neural network to sequentially generate pieces of an image. We evaluate the effect of layer normalization on a DRAW model using 64 glimpses and 256 LSTM hidden units. The model is trained with the default setting of Adam [Kingma and Ba, 2014] optimizer and the minibatch size of 128. Previous publications on binarized MNIST have used various training protocols to generate their datasets. In this experiment, we used the fixed binarization from Larochelle and Murray [2011]. The dataset has been split into 50,000 training, 10,000 validation and 10,000 test images.

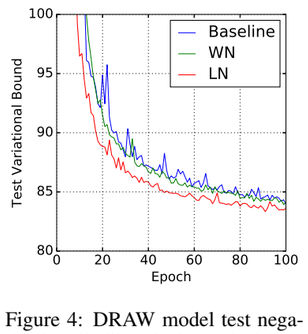


Figure 4: DRAW model test negative log likelihood with and without layer normalization.

Figure 4 shows the test variational bound for the first 100 epoch. It highlights the speedup benefit of applying layer normalization that the layer normalized DRAW converges almost twice as fast than the baseline model. After 200 epoches, the baseline model converges to a variational log likelihood of 82.36 nats on the test data and the layer normalization model obtains 82.09 nats.

6.5 Handwriting sequence generation

The previous experiments mostly examine RNNs on NLP tasks whose lengths are in the range of 10 to 40. To show the effectiveness of layer normalization on longer sequences, we performed handwriting generation tasks using the IAM Online Handwriting Database [Liwicki and Bunke, 2005]. IAM-OnDB consists of handwritten lines collected from 221 different writers. When given the input character string, the goal is to predict a sequence of x and y pen co-ordinates of the corresponding handwriting line on the whiteboard. There are, in total, 12179 handwriting line sequences. The input string is typically more than 25 characters and the average handwriting line has a length around 700.

We used the same model architecture as in Section (5.2) of Graves [2013]. The model architecture consists of three hidden layers of 400 LSTM cells, which produce 20 bivariate Gaussian mixture components at the output layer, and a size 3 input layer. The character sequence was encoded with one-hot vectors, and hence the window vectors were size 57. A mixture of 10 Gaussian functions was used for the window parameters, requiring a size 30 parameter vector. The total number of weights was increased to approximately 3.7M. The model is trained using mini-batches of size 8 and the Adam [Kingma and Ba, 2014] optimizer.

The combination of small mini-batch size and very long sequences makes it important to have very stable hidden dynamics. Figure 5 shows that layer normalization converges to a comparable log likelihood as the baseline model but is much faster.

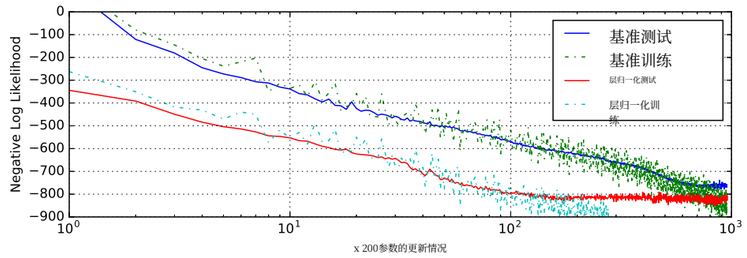


图5：采用层归一化与未采用层归一化的手写序列生成模型的负对数似然值。这些模型是在小批量大小为8、序列长度为500的条件下进行训练的。

现有研究结果与我们的结果之间存在差异，其原因很可能在于公开可用的代码并未像原始模型那样对解码器的每个时间步进行条件处理。

6.4 使用DRAW算法对二值化后的MNIST数据集进行建模

We also experimented with the generative modeling on the MNIST dataset. Deep Recurrent Attention Writer (DRAW) [Gregor et al., 2015] has previously achieved the state-of-the-art performance on modeling the distribution of MNIST digits. The model uses a differential attention mechanism and a recurrent neural network to sequentially generate pieces of an image. We evaluate the effect of layer normalization on a DRAW model using 64 glimpses and 256 LSTM hidden units. The model is trained with the default setting of Adam [Kingma and Ba, 2014] optimizer and the minibatch size of 128. Previous publications on binarized MNIST have used various training protocols to generate their datasets. In this experiment, we used the fixed binarization from Larochelle and Murray [2011]. The dataset has been split into 50,000 training, 10,000 validation and 10,000 test images.

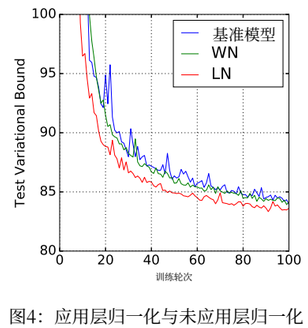


图4：应用层归一化与未应用层归一化时的DRAW模型测试负对数似然值。

图4展示了前100个训练轮次的测试变差界，突显了应用层归一化所带来的加速优势。
研究表明，应用了层归一化的DRAW算法的收敛速度几乎是以基准模型为两倍。在经过200个训练轮次后，基准模型在测试数据上的变分对数似然值达到82.36纳特，而采用层归一化的模型则达到了82.09纳特。

6.5 手写序列生成

之前的实验大多是在长度在10到40之间的自然语言处理任务上测试循环神经网络的效果。为了验证层归一化在更长序列上的有效性，我们使用了IAM在线手写数据库 [Liwicki和Bunke, 2005]来开展手写生成任务。该数据库包含了来自221位不同书写者的手写线条。给定输入的字符字符串后，系统的目标是预测白板上对应手写线条的x轴和y轴坐标序列。整个数据库中共包含12179条手写线条序列，输入字符串的长度通常超过25个字符，而平均每条手写线条的长度约为700。

我们采用了Graves [2013]第(5.2)节中相同的模型架构。该模型架构包含三层隐藏层，每层各有400个LSTM单元；在输出层则生成20个双变量高斯混合分量，同时还设有一个规模为3的输入层。字符序列通过独热向量进行编码，因此窗口向量的维度为57。窗口参数采用10个高斯函数的混合形式，这需要一个规模为30的参数向量。整个模型的权重总数大约达到了370万。该模型使用规模为8的小批量数据，并结合Adam [Kingma和Ba, 2014] 优化器进行训练。

由于小批量数据大小与序列长度都较长，因此保持隐藏层动态的极度稳定显得尤为重要。图5显示，层归一化能够收敛到与基准模型相当的对数似然值，且收敛速度要快得多。

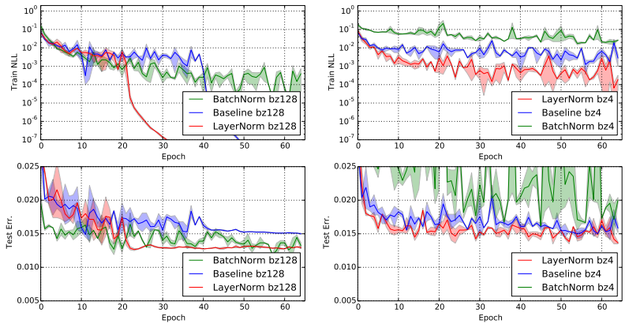


Figure 6: Permutation invariant MNIST 784-1000-1000-10 model negative log likelihood and test error with layer normalization and batch normalization. (Left) The models are trained with batch-size of 128. (Right) The models are trained with batch-size of 4.

6.6 Permutation invariant MNIST

In addition to RNNs, we investigated layer normalization in feed-forward networks. We show how layer normalization compares with batch normalization on the well-studied permutation invariant MNIST classification problem. From the previous analysis, layer normalization is invariant to input re-scaling which is desirable for the internal hidden layers. But this is unnecessary for the logit outputs where the prediction confidence is determined by the scale of the logits. We only apply layer normalization to the fully-connected hidden layers that excludes the last softmax layer.

All the models were trained using 55000 training data points and the Adam [Kingma and Ba, 2014] optimizer. For the smaller batch-size, the variance term for batch normalization is computed using the unbiased estimator. The experimental results from Figure 6 highlight that layer normalization is robust to the batch-sizes and exhibits a faster training convergence comparing to batch normalization that is applied to all layers.

6.7 Convolutional Networks

We have also experimented with convolutional neural networks. In our preliminary experiments, we observed that layer normalization offers a speedup over the baseline model without normalization, but batch normalization outperforms the other methods. With fully connected layers, all the hidden units in a layer tend to make similar contributions to the final prediction and re-centering and re-scaling the summed inputs to a layer works well. However, the assumption of similar contributions is no longer true for convolutional neural networks. The large number of the hidden units whose receptive fields lie near the boundary of the image are rarely turned on and thus have very different statistics from the rest of the hidden units within the same layer. We think further research is needed to make layer normalization work well in ConvNets.

7 Conclusion

In this paper, we introduced layer normalization to speed-up the training of neural networks. We provided a theoretical analysis that compared the invariance properties of layer normalization with batch normalization and weight normalization. We showed that layer normalization is invariant to per training-case feature shifting and scaling.

Empirically, we showed that recurrent neural networks benefit the most from the proposed method especially for long sequences and small mini-batches.

Acknowledgments

This research was funded by grants from NSERC, CFI, and Google.

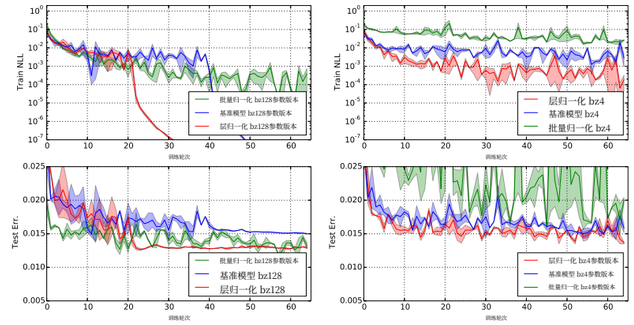


图6：采用层归一化与批量归一化的具有不变量特性的MNIST 784-1000-1000-10模型的负对数似然值与测试误差曲线。（左图）这些模型是在批次大小为128的条件下进行训练的；（右图）这些模型则是在批次大小为4的条件下进行训练的。

6.6 具有不变性特性的MNIST

除了循环神经网络之外，我们还研究了前馈神经网络中的层归一化技术。通过在经过深入研究的、具有排列不变性的MNIST分类问题上进行对比，我们展示了层归一化与批量归一化的表现差异。根据之前的分析，层归一化对输入数据的重新缩放具有不变性，这一特性对于内部隐藏层而言十分理想；但对于用于确定预测置信度的logit输出来说，这种不变性则并无必要。因此，我们仅将层归一化应用于那些全连接型的隐藏层，而不包括最后的softmax层。

所有模型均使用55000个训练数据点以及Adam [Kingma和Ba, 2014]优化器进行训练。在较小的批量大小下，批量归一化的方差项是采用无偏估计量来计算的。图6中的实验结果表明，层归一化对批量大小具有很强的鲁棒性，且与应用于所有层的批量归一化相比，其训练收敛速度更快。

6.7 卷积神经网络

我们还对卷积神经网络进行了实验。在初步试验中，我们发现层归一化相比不进行归一化的基准模型能够提升速度，而批量归一化的表现则优于其他方法。在全连接层中，同一层内的所有隐藏单元往往会对最终预测产生相似的贡献，因此将该层内各输入值重新居中并重新缩放是一种有效的方法。然而，这一关于贡献相似性的假设在卷积神经网络中已不再成立。由于大量接收场位于图像边缘附近的隐藏单元很少被激活，其统计特性与同一层内其他隐藏单元存在极大差异。我们认为需要进一步的研究才能让层归一化在卷积神经网络中也能发挥良好作用。

7 结论

在本文中，我们引入了层归一化技术，用以加速神经网络的训练过程。同时，我们还进行了理论分析，对比了层归一化与批量归一化以及权重归一化的不变性特征，并证明层归一化对针对不同训练案例的特征平移和缩放操作具备不变性。

通过实证研究，我们发现循环神经网络从所提出的方法中获益最大，尤其是在处理长序列以及小批量数据时效果更为显著。

致谢

本研究 得到了加拿大自然科学与工程研究委员会、加拿大创新基金会等机构的资助。 谷歌。

References

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.

Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE*, 2012.

Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, et al. Large scale distributed deep networks. In *NIPS*, 2012.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*, 2015.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014.

César Laurent, Gabriel Pereyra, Philémon Brakel, Ying Zhang, and Yoshua Bengio. Batch normalized recurrent neural networks. *arXiv preprint arXiv:1510.01378*, 2015.

Dario Amodei, Rishita Anubhai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski, Adam Coates, Greg Diamos, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. *arXiv preprint arXiv:1512.02595*, 2015.

Tim Cooijmans, Nicolas Ballas, César Laurent, and Aaron Courville. Recurrent batch normalization. *arXiv preprint arXiv:1603.09025*, 2016.

Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *arXiv preprint arXiv:1602.07868*, 2016.

Behnam Neyshabur, Ruslan R Salakhutdinov, and Nati Srebro. Path-sgd: Path-normalized optimization in deep neural networks. In *Advances in Neural Information Processing Systems*, pages 2413–2421, 2015.

Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural computation*, 1998.

Ivan Vendrov, Ryan Kiros, Sanja Fidler, and Raquel Urtasun. Order-embeddings of images and language. *ICLR*, 2016.

The Theano Development Team, Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermueller, Dzmitry Bahdanau, Nicolas Ballas, Frédéric Bastien, Justin Bayer, Anatoly Belikov, et al. Theano: A python framework for fast computation of mathematical expressions. *arXiv preprint arXiv:1605.02688*, 2016.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. *ECCV*, 2014.

Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *EMNLP*, 2014.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *ICLR*, 2015.

Ryan Kiros, Ruslan Salakhutdinov, and Richard S Zemel. Unifying visual-semantic embeddings with multimodal neural language models. *arXiv preprint arXiv:1411.2539*, 2014.

D. Kingma and J. L. Ba. Adam: a method for stochastic optimization. *ICLR*, 2014. arXiv:1412.6980.

Liwei Wang, Yin Li, and Svetlana Lazebnik. Learning deep structure-preserving image-text embeddings. *CVPR*, 2016.

Karl Moritz Hermann, Tomas Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *NIPS*, 2015.

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Skip-thought vectors. In *NIPS*, 2015.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *ICCV*, 2015.

Marco Marelli, Luisa Bentivogli, Marco Baroni, Raffaella Bernardi, Stefano Menini, and Roberto Zamparelli. Semeval-2014 task 1: Evaluation of compositional distributional semantic models on full sentences through semantic relatedness and textual entailment. *SemEval-2014*, 2014.

参考文献

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.

Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE*, 2012.

Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, et al. Large scale distributed deep networks. In *NIPS*, 2012.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*, 2015.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014.

César Laurent, Gabriel Pereyra, Philémon Brakel, Ying Zhang, and Yoshua Bengio. Batch normalized recurrent neural networks. *arXiv preprint arXiv:1510.01378*, 2015.

Dario Amodei, Rishita Anubhai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski, Adam Coates, Greg Diamos, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. *arXiv preprint arXiv:1512.02595*, 2015.

Tim Cooijmans, Nicolas Ballas, César Laurent, and Aaron Courville. Recurrent batch normalization. *arXiv preprint arXiv:1603.09025*, 2016.

Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *arXiv preprint arXiv:1602.07868*, 2016.

Behnam Neyshabur, Ruslan R Salakhutdinov, and Nati Srebro. Path-sgd: Path-normalized optimization in deep neural networks. In *Advances in Neural Information Processing Systems*, pages 2413–2421, 2015.

Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural computation*, 1998.

Ivan Vendrov, Ryan Kiros, Sanja Fidler, and Raquel Urtasun. Order-embeddings of images and language. *ICLR*, 2016.

The Theano Development Team, Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermueller, Dzmitry Bahdanau, Nicolas Ballas, Frédéric Bastien, Justin Bayer, Anatoly Belikov, et al. Theano: A python framework for fast computation of mathematical expressions. *arXiv preprint arXiv:1605.02688*, 2016.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. *ECCV*, 2014.

Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *EMNLP*, 2014.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *ICLR*, 2015.

Ryan Kiros, Ruslan Salakhutdinov, and Richard S Zemel. Unifying visual-semantic embeddings with multimodal neural language models. *arXiv preprint arXiv:1411.2539*, 2014.

D. Kingma and J. L. Ba. Adam: a method for stochastic optimization. *ICLR*, 2014. arXiv:1412.6980.

Liwei Wang, Yin Li, and Svetlana Lazebnik. Learning deep structure-preserving image-text embeddings. *CVPR*, 2016.

Karl Moritz Hermann, Tomas Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *NIPS*, 2015.

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Skip-thought vectors. In *NIPS*, 2015.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *ICCV*, 2015.

Marco Marelli, Luisa Bentivogli, Marco Baroni, Raffaella Bernardi, Stefano Menini, and Roberto Zamparelli. Semeval-2014 task 1: Evaluation of compositional distributional semantic models on full sentences through semantic relatedness and textual entailment. *SemEval-2014*, 2014.

Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *ACL*, pages 115–124, 2005.

Minqing Hu and Bing Liu. Mining and summarizing customer reviews. In *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, 2004.

Bo Pang and Lillian Lee. A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts. In *ACL*, 2004.

Janyce Wiebe, Theresa Wilson, and Claire Cardie. Annotating expressions of opinions and emotions in language. *Language resources and evaluation*, 2005.

K. Gregor, I. Danihelka, A. Graves, and D. Wierstra. DRAW: a recurrent neural network for image generation. arXiv:1502.04623, 2015.

Hugo Larochelle and Iain Murray. The neural autoregressive distribution estimator. In *AISTATS*, volume 6, page 622, 2011.

Marcus Liwicki and Horst Bunke. Iam-ondb-an on-line english sentence database acquired from handwritten text on a whiteboard. In *ICDAR*, 2005.

Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *ACL*, pages 115–124, 2005.

Minqing Hu and Bing Liu. Mining and summarizing customer reviews. In *Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*, 2004.

Bo Pang and Lillian Lee. A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts. In *ACL*, 2004.

Janyce Wiebe, Theresa Wilson, and Claire Cardie. Annotating expressions of opinions and emotions in language. *Language resources and evaluation*, 2005.

K. Gregor, I. Danihelka, A. Graves, and D. Wierstra. DRAW: a recurrent neural network for image generation. arXiv:1502.04623, 2015.

Hugo Larochelle and Iain Murray. The neural autoregressive distribution estimator. In *AISTATS*, volume 6, page 622, 2011.

Marcus Liwicki and Horst Bunke. Iam-ondb-an on-line english sentence database acquired from handwritten text on a whiteboard. In *ICDAR*, 2005.

Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

12

Supplementary Material

Application of layer normalization to each experiment

This section describes how layer normalization is applied to each of the papers' experiments. For notation convenience, we define layer normalization as a function mapping $LN : \mathbb{R}^D \to \mathbb{R}^D$ with two set of adaptive parameters, gains $\boldsymbol{\alpha}$ and biases $\boldsymbol{\beta}$:

$$LN(\mathbf{z}; \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{(\mathbf{z} - \mu)}{\sigma} \odot \boldsymbol{\alpha} + \boldsymbol{\beta}, \tag{15}$$

$$\mu = \frac{1}{D}\sum_{i=1}^{D} z_i, \quad \sigma = \sqrt{\frac{1}{D}\sum_{i=1}^{D}(z_i - \mu)^2}, \tag{16}$$

where, z_i is the ith element of the vector $\mathbf{z}$.

Teaching machines to read and comprehend and handwriting sequence generation

The basic LSTM equations used for these experiment are given by:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b \tag{17}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{18}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(\mathbf{c}_t) \tag{19}$$

The version that incorporates layer normalization is modified as follows:

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \boldsymbol{\alpha}_1, \boldsymbol{\beta}_1) + LN(\mathbf{W}_x \mathbf{x}_t; \boldsymbol{\alpha}_2, \boldsymbol{\beta}_2) + b \tag{20}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{21}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(LN(\mathbf{c}_t; \boldsymbol{\alpha}_3, \boldsymbol{\beta}_3)) \tag{22}$$

where $\boldsymbol{\alpha}_i, \boldsymbol{\beta}_i$ are the additive and multiplicative parameters, respectively. Each $\boldsymbol{\alpha}_i$ is initialized to a vector of zeros and each $\boldsymbol{\beta}_i$ is initialized to a vector of ones.

Order embeddings and skip-thoughts

These experiments utilize a variant of gated recurrent unit which is defined as follows:

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t \tag{23}$$

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \sigma(\mathbf{r}_t) \odot (\mathbf{U}\mathbf{h}_{t-1})) \tag{24}$$

$$\mathbf{h}_t = (1 - \sigma(\mathbf{z}_t))\mathbf{h}_{t-1} + \sigma(\mathbf{z}_t)\hat{\mathbf{h}}_t \tag{25}$$

Layer normalization is applied as follows:

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \boldsymbol{\alpha}_1, \boldsymbol{\beta}_1) + LN(\mathbf{W}_x \mathbf{x}_t; \boldsymbol{\alpha}_2, \boldsymbol{\beta}_2) \tag{26}$$

$$\hat{\mathbf{h}}_t = \tanh(LN(\mathbf{W}\mathbf{x}_t; \boldsymbol{\alpha}_3, \boldsymbol{\beta}_3) + \sigma(\mathbf{r}_t) \odot LN(\mathbf{U}\mathbf{h}_{t-1}; \boldsymbol{\alpha}_4, \boldsymbol{\beta}_4)) \tag{27}$$

$$\mathbf{h}_t = (1 - \sigma(\mathbf{z}_t))\mathbf{h}_{t-1} + \sigma(\mathbf{z}_t)\hat{\mathbf{h}}_t \tag{28}$$

just as before, $\boldsymbol{\alpha}_i$ is initialized to a vector of zeros and each $\boldsymbol{\beta}_i$ is initialized to a vector of ones.

补充材料

层归一化在各项实验中的应用

本节介绍了层归一化是如何应用于各篇论文中的实验的。为便于表述，我们将层归一化定义为一个映射函数 LN：$\mathbb{R}^D \to \mathbb{R}^D$，该函数包含两组自适应参数：增益 $\boldsymbol{\alpha}$ 和偏置 $\boldsymbol{\beta}$。

$$LN(\mathbf{z}; \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{(\mathbf{z} - \mu)}{\sigma} \odot \boldsymbol{\alpha} + \boldsymbol{\beta}, \tag{15}$$

$$\mu = \frac{1}{D}\sum_{i=1}^{D} z_i, \quad \sigma = \sqrt{\frac{1}{D}\sum_{i=1}^{D}(z_i - \mu)^2}, \tag{16}$$

其中， z_i 为向量$\mathbf{z}$的 i^{th} 分量。

教学机器实现阅读与理解以及手写序列生成

这些实验所使用的基本LSTM方程如下：

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + b \tag{17}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{18}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(\mathbf{c}_t) \tag{19}$$

引入层归一化的版本经过如下修改：

$$\begin{pmatrix} \mathbf{f}_t \\ \mathbf{i}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \boldsymbol{\alpha}_1, \boldsymbol{\beta}_1) + LN(\mathbf{W}_x \mathbf{x}_t; \boldsymbol{\alpha}_2, \boldsymbol{\beta}_2) + b \tag{20}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t) \odot \mathbf{c}_{t-1} + \sigma(\mathbf{i}_t) \odot \tanh(\mathbf{g}_t) \tag{21}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t) \odot \tanh(LN(\mathbf{c}_t; \boldsymbol{\alpha}_3, \boldsymbol{\beta}_3)) \tag{22}$$

其中 $\boldsymbol{\alpha}_i$， $\boldsymbol{\beta}_i$ 分别代表加法参数和乘法参数。每个 $\boldsymbol{\alpha}_i$ 都会被初始化为一个全零向量，而每个 $\boldsymbol{\beta}_i$ 则会被初始化为一个全一向量。

顺序嵌入与跳跃思维

这些实验采用了一种改进版的门控循环单元，其定义如下：

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t \tag{23}$$

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \sigma(\mathbf{r}_t) \odot (\mathbf{U}\mathbf{h}_{t-1})) \tag{24}$$

$$\mathbf{h}_t = (1 - \sigma(\mathbf{z}_t))\mathbf{h}_{t-1} + \sigma(\mathbf{z}_t)\hat{\mathbf{h}}_t \tag{25}$$

层归一化的应用方式如下：

$$\begin{pmatrix} \mathbf{z}_t \\ \mathbf{r}_t \end{pmatrix} = LN(\mathbf{W}_h \mathbf{h}_{t-1}; \boldsymbol{\alpha}_1, \boldsymbol{\beta}_1) + LN(\mathbf{W}_x \mathbf{x}_t; \boldsymbol{\alpha}_2, \boldsymbol{\beta}_2) \tag{26}$$

$$\hat{\mathbf{h}}_t = \tanh(LN(\mathbf{W}\mathbf{x}_t; \boldsymbol{\alpha}_3, \boldsymbol{\beta}_3) + \sigma(\mathbf{r}_t) \odot LN(\mathbf{U}\mathbf{h}_{t-1}; \boldsymbol{\alpha}_4, \boldsymbol{\beta}_4)) \tag{27}$$

$$\mathbf{h}_t = (1 - \sigma(\mathbf{z}_t))\mathbf{h}_{t-1} + \sigma(\mathbf{z}_t)\hat{\mathbf{h}}_t \tag{28}$$

j与之前相同， $\boldsymbol{\alpha}_i$ 会被初始化为一个全零向量，而每个 $\boldsymbol{\beta}_i$ 则会被初始化为一个全一向量。

Modeling binarized MNIST using DRAW

The layer norm is only applied to the output of the LSTM hidden states in this experiment:

The version that incorporates layer normalization is modified as follows:

$$\begin{pmatrix}\mathbf{f}_t\\ \mathbf{i}_t\\ \mathbf{o}_t\\ \mathbf{g}_t\end{pmatrix} = \mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_x\mathbf{x}_t + b \tag{29}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t)\odot\mathbf{c}_{t-1} + \sigma(\mathbf{i}_t)\odot\tanh(\mathbf{g}_t) \tag{30}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t)\odot\tanh(LN(\mathbf{c}_t;\boldsymbol{\alpha},\boldsymbol{\beta})) \tag{31}$$

where $\boldsymbol{\alpha},\boldsymbol{\beta}$ are the additive and multiplicative parameters, respectively. $\boldsymbol{\alpha}$ is initialized to a vector of zeros and $\boldsymbol{\beta}$ is initialized to a vector of ones.

Learning the magnitude of incoming weights

We now compare how gradient descent updates changing magnitude of the equivalent weights between the normalized GLM and original parameterization. The magnitude of the weights are explicitly parameterized using the gain parameter in the normalized model. Assume there is a gradient update that changes norm of the weight vectors by δ_g. We can project the gradient updates to the weight vector for the normal GLM. The KL metric, ie how much the gradient update changes the model prediction, for the normalized model depends only on the magnitude of the prediction error. Specifically,

under batch normalization:

$$ds^2 = \frac{1}{2}\operatorname{vec}([0,0,\delta_g]^\top)^\top\bar{F}(\operatorname{vec}([W,\mathbf{b},\mathbf{g}]^\top)\operatorname{vec}([0,0,\delta_g]^\top) = \frac{1}{2}\delta_g^\top\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\operatorname{Cov}[\mathbf{y}\mid\mathbf{x}]}{\phi^2}\right]\delta_g. \tag{32}$$

Under layer normalization:

$$\begin{aligned}ds^2 =& \frac{1}{2}\operatorname{vec}([0,0,\delta_g]^\top)^\top\bar{F}(\operatorname{vec}([W,\mathbf{b},\mathbf{g}]^\top)\operatorname{vec}([0,0,\delta_g]^\top)\\ =& \frac{1}{2}\delta_g^\top\frac{1}{\phi^2}\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\begin{bmatrix}\operatorname{Cov}(y_1,y_1\mid\mathbf{x})\frac{(a_1-\mu)^2}{\sigma^2} & \cdots & \operatorname{Cov}(y_1,y_H\mid\mathbf{x})\frac{(a_1-\mu)(a_H-\mu)}{\sigma^2}\\ \vdots & \ddots & \vdots\\ \operatorname{Cov}(y_H,y_1\mid\mathbf{x})\frac{(a_H-\mu)(a_1-\mu)}{\sigma^2} & \cdots & \operatorname{Cov}(y_H,y_H\mid\mathbf{x})\frac{(a_H-\mu)^2}{\sigma^2}\end{bmatrix}\delta_g\end{aligned} \tag{33}$$

Under weight normalization:

$$\begin{aligned}ds^2 =& \frac{1}{2}\operatorname{vec}([0,0,\delta_g]^\top)^\top\bar{F}(\operatorname{vec}([W,\mathbf{b},\mathbf{g}]^\top)\operatorname{vec}([0,0,\delta_g]^\top)\\ =& \frac{1}{2}\delta_g^\top\frac{1}{\phi^2}\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\begin{bmatrix}\operatorname{Cov}(y_1,y_1\mid\mathbf{x})\frac{a_1^2}{\|w_1\|_2^2} & \cdots & \operatorname{Cov}(y_1,y_H\mid\mathbf{x})\frac{a_1a_H}{\|w_1\|_2\|w_H\|_2}\\ \vdots & \ddots & \vdots\\ \operatorname{Cov}(y_H,y_1\mid\mathbf{x})\frac{a_Ha_1}{\|w_H\|_2\|w_1\|_2} & \cdots & \operatorname{Cov}(y_H,y_H\mid\mathbf{x})\frac{a_H^2}{\|w_H\|_2^2}\end{bmatrix}\delta_g.\end{aligned} \tag{34}$$

Whereas, the KL metric in the standard GLM is related to its activities $a_i = w_i^\top\mathbf{x}$, that is depended on both its current weights and input data. We project the gradient updates to the gain parameter $\delta_g i$ of the ith neuron to its weight vector as $\delta_{gi}\frac{w_i}{\|w_i\|_2}$ in the standard GLM model:

$$\begin{aligned}&\frac{1}{2}\operatorname{vec}([\delta_{gi}\frac{w_i}{\|w_i\|_2},0,\delta_{gj}\frac{w_j}{\|w_i\|_2},0]^\top)^\top F([w_i^\top,b_i,w_j^\top,b_j]^\top)\operatorname{vec}([\delta_{gi}\frac{w_i}{\|w_i\|_2},0,\delta_{gj}\frac{w_j}{\|w_j\|_2},0]^\top)\\ =&\frac{\delta_{gi}\delta_{gj}}{2\phi^2}\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\operatorname{Cov}(y_i,y_j\mid\mathbf{x})\frac{a_ia_j}{\|w_i\|_2\|w_j\|_2}\right]\end{aligned} \tag{35}$$

The batch normalized and layer normalized models are therefore more robust to the scaling of the input and its parameters than the standard model.

在本实验中，层归一化仅应用于LSTM隐藏状态的输出结果上。

加入了层归一化的版本经过如下修改：

$$\begin{pmatrix}\mathbf{f}_t\\ \mathbf{i}_t\\ \mathbf{o}_t\\ \mathbf{g}_t\end{pmatrix} = \mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_x\mathbf{x}_t + b \tag{29}$$

$$\mathbf{c}_t = \sigma(\mathbf{f}_t)\odot\mathbf{c}_{t-1} + \sigma(\mathbf{i}_t)\odot\tanh(\mathbf{g}_t) \tag{30}$$

$$\mathbf{h}_t = \sigma(\mathbf{o}_t)\odot\tanh(LN(\mathbf{c}_t;\boldsymbol{\alpha},\boldsymbol{\beta})) \tag{31}$$

其中 $\boldsymbol{\alpha}$， $\boldsymbol{\beta}$ 分别代表加法参数与乘法参数。 $\boldsymbol{\alpha}$ 会被初始化为全零向量，而 $\boldsymbol{\beta}$ 则会被初始化为全一向量。

学习过程：确定输入权重的幅值

接下来我们将对比在归一化后的GLM模型与原始参数化结构中，梯度下降法如何调整对应权重幅值的变化。在归一化模型中，权重的幅值是通过增益参数来明确设定的。假设某次梯度更新使权重向量的范数发生变化 δ_g，我们就可以将该梯度更新映射到普通GLM的权重向量上。对于归一化模型而言，KL散度——即梯度更新对模型预测结果影响的大小——仅取决于预测误差的幅值，具体而言

在批量归一化环境下：

$$ds^2 = \frac{1}{2}\operatorname{vec}([0,0,\delta_g]^\top)^\top\bar{F}(\operatorname{vec}([W,\mathbf{b},\mathbf{g}]^\top)\operatorname{vec}([0,0,\delta_g]^\top) = \frac{1}{2}\delta_g^\top\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\operatorname{Cov}[\mathbf{y}\mid\mathbf{x}]}{\phi^2}\right]\delta_g. \tag{32}$$

在层归一化的背景下：

$$\begin{aligned}ds^2 =& \frac{1}{2}\operatorname{vec}([0,0,\delta_g]^\top)^\top\bar{F}(\operatorname{vec}([W,\mathbf{b},\mathbf{g}]^\top)\operatorname{vec}([0,0,\delta_g]^\top)\\ =& \frac{1}{2}\delta_g^\top\frac{1}{\phi^2}\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\begin{bmatrix}\operatorname{Cov}(y_1,y_1\mid\mathbf{x})\frac{(a_1-\mu)^2}{\sigma^2} & \cdots & \operatorname{Cov}(y_1,y_H\mid\mathbf{x})\frac{(a_1-\mu)(a_H-\mu)}{\sigma^2}\\ \vdots & \ddots & \vdots\\ \operatorname{Cov}(y_H,y_1\mid\mathbf{x})\frac{(a_H-\mu)(a_1-\mu)}{\sigma^2} & \cdots & \operatorname{Cov}(y_H,y_H\mid\mathbf{x})\frac{(a_H-\mu)^2}{\sigma^2}\end{bmatrix}\delta_g\end{aligned} \tag{33}$$

在权重归一化的背景下：

$$\begin{aligned}ds^2 =& \frac{1}{2}\operatorname{vec}([0,0,\delta_g]^\top)^\top\bar{F}(\operatorname{vec}([W,\mathbf{b},\mathbf{g}]^\top)\operatorname{vec}([0,0,\delta_g]^\top)\\ =& \frac{1}{2}\delta_g^\top\frac{1}{\phi^2}\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\begin{bmatrix}\operatorname{Cov}(y_1,y_1\mid\mathbf{x})\frac{a_1^2}{\|w_1\|_2^2} & \cdots & \operatorname{Cov}(y_1,y_H\mid\mathbf{x})\frac{a_1a_H}{\|w_1\|_2\|w_H\|_2}\\ \vdots & \ddots & \vdots\\ \operatorname{Cov}(y_H,y_1\mid\mathbf{x})\frac{a_Ha_1}{\|w_H\|_2\|w_1\|_2} & \cdots & \operatorname{Cov}(y_H,y_H\mid\mathbf{x})\frac{a_H^2}{\|w_H\|_2^2}\end{bmatrix}\delta_g.\end{aligned} \tag{34}$$

而在标准GLM模型中，KL散度与其激活值 $a_i = w_i^\top\mathbf{x}$相关，该激活值同时取决于模型的当前权重和输入数据。在标准GLM模型中，我们会将梯度更新按照 $\delta_{gi}\frac{w_i}{\|w_i\|_2}$ 的方式，从 i^{th} 神经元的增益参数 $\delta_g i$映射到其权重向量上。

$$\begin{aligned}&\frac{1}{2}\operatorname{vec}([\delta_{gi}\frac{w_i}{\|w_i\|_2},0,\delta_{gj}\frac{w_j}{\|w_i\|_2},0]^\top)^\top F([w_i^\top,b_i,w_j^\top,b_j]^\top)\operatorname{vec}([\delta_{gi}\frac{w_i}{\|w_i\|_2},0,\delta_{gj}\frac{w_j}{\|w_j\|_2},0]^\top)\\ =&\frac{\delta_{gi}\delta_{gj}}{2\phi^2}\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\operatorname{Cov}(y_i,y_j\mid\mathbf{x})\frac{a_ia_j}{\|w_i\|_2\|w_j\|_2}\right]\end{aligned} \tag{35}$$

因此，经过批量归一化和层归一化处理的模型，相比标准模型而言，对输入数据及其参数的尺度变化具有更强的鲁棒性。